«Код Будущего — программирование на Python для специалистов по информационной безопасности»

Модуль 2. Файлы, данные, простая обработка и визуализация.

ТЕМА Классы и объекты. Наследование, инкапсуляция и полиморфизм. Создание класса обработки уязвимостей

Практическое занятие

Цель:

Научиться создавать классы под свои нужды.

Задачи:

Узнаем что такое класс

Как создать свой класс

Какие у классов есть свойства.

Методические указания и инструкции по выполнению задания

1. **Вспомните и повторите теорию, необходимую для выполнения практического задания**

ООП в Python: Классы — шаблоны для объектов (атрибуты + методы).

Основные принципы:

Инкапсуляция: Сокрытие данных (self._private).

Наследование: Класс-потомок (class Child(Parent)).

Полиморфизм: Перегрузка операторов (__add__, __str__).

Пример: Класс для обработки уязвимостей (атрибуты: CVE, CVSS; методы: анализ риска).

2. **Разберитесь в том, как выполняется задание**

Пример программы с классом для представления пользователя.

Задание 1

Напишите класс ученика класса. Опишите в классе все возможные свойства, которые можно обобщить для каждого ученика, а также создать несколько методов.

Описание для тренажера:

1. Создать класс `Student`, который описывает ученика.
2. В конструкторе `__init__` определить свойства: имя, возраст, класс (например, 10A) и список оценок.
3. Создать метод `add_mark`, который добавляет оценку в список оценок.
4. Создать метод `get_average_mark`, который возвращает среднюю оценку ученика (если оценок нет, возвращать 0).
5. Создать метод `get_info`, который возвращает строку с информацией о ученике в формате: имя, возраст, класс.
6. Класс должен содержать хотя бы один метод и хотя бы один атрибут экземпляра.
7. Не использовать запрещённые конструкции: import, eval, exec, subprocess.

Тренажер 20501-student-class

Решение:

```
class Student:
    def __init__(self, name, age, grade):
        self.name = name        # имя
        self.age = age          # возраст
        self.grade = grade      # класс (10A)
        self.marks = []         # список оценок

    def add_mark(self, mark):
        self.marks.append(mark)

    def get_average_mark(self):
        if not self.marks:
            return 0
        return sum(self.marks) / len(self.marks)

    def get_info(self):
        return f"{self.name}, {self.age}, {self.grade}"
```

Задание 2

Переопределите метод len для нахождения длины списка/**строки**, чтобы он выводил всегда число 25

Описание для тренажера:

1. Создать класс `Always25`.
2. В классе переопределить метод `__len__`.
3. Метод `__len__` должен всегда возвращать число 25, независимо от содержимого объекта.
4. Не использовать запрещённые конструкции: import, eval, exec, subprocess.
5. Проверить, что при вызове `len()` для экземпляра класса возвращается 25.

Тренажер **20502-finding-the-length**

Решение:

```
class Always25:
    def __len__(self):
        return 25
```

Задание 3

Реализовать класс и переопределить методы базовых математических операций (сложение, вычитание, умножение, деление), добавив туда выводы в консоль текущего действия.

Например: при умножении выводится сообщение, что происходит умножение.

Описание для тренажера:

1. Создать класс `VerboseCalculator`, который хранит числовое значение.

2. В конструкторе `__init__` определить свойство `value`, которое хранит текущее значение объекта.

3. Переопределить методы арифметических операций: `__add__`, `__sub__`, `__mul__`, `__truediv__`.

4. В каждом методе перед выполнением операции выводить сообщение о текущем действии, например: «Выполняется сложение», «Выполняется вычитание», «Выполняется умножение», «Выполняется деление».

5. В методе деления предусмотреть проверку деления на ноль. Если деление на ноль, вывести сообщение «Ошибка: деление на ноль» и вернуть объект с `value` равным `nan`.

6. Методы должны возвращать новый объект класса `VerboseCalculator` с результатом операции.

7. Переопределить метод `__str__`, чтобы при выводе объекта отображалось его значение.

8. Не использовать запрещённые конструкции: import, eval, exec, subprocess.
9.

Тренажер **20503-VerboseCalculator**

Пример корректного решения:

```
class VerboseCalculator:
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        print("Выполняется сложение")
        return VerboseCalculator(self.value + other.value)

    def __sub__(self, other):
        print("Выполняется вычитание")
        return VerboseCalculator(self.value - other.value)

    def __mul__(self, other):
        print("Выполняется умножение")
        return VerboseCalculator(self.value * other.value)

    def __truediv__(self, other):
```

```
        print("Выполняется деление")
        if other.value == 0:
            print("Ошибка: деление на ноль")
            return VerboseCalculator(float('nan'))
        return VerboseCalculator(self.value / other.value)

    def __str__(self):
        return str(self.value)
# Пример использования
a = VerboseCalculator(10)
b = VerboseCalculator(2)
print(a + b)  # Выполняется сложение → 12
print(a * b)  # Выполняется умножение → 20
print(a / b)  # Выполняется деление → 5.0
print(a / VerboseCalculator(0))  # Выполняется деление → Ошибка →
nan
```

Задание 4

Реализуйте класс для хранения результатов анализа файла vulnreport.json

Описание для тренажера:

1. Создать класс `VulnReport`, который хранит результаты анализа файла vulnreport.json.
2. В конструкторе `__init__` загрузить данные из JSON-файла и сохранить их в атрибут `data`.
3. Если файла нет, вывести сообщение: Файл vulnreport.json не найден.
4. Если файл не является корректным JSON, вывести сообщение: Ошибка чтения JSON файла.
5. Реализовать метод `count_vulnerabilities`, который

возвращает количество уязвимостей в данных.

6. Реализовать метод `list_ips`, который возвращает список всех IP-адресов, если они есть в данных.

7. Реализовать метод `highest_cvss`, который возвращает запись с самой высокой оценкой CVSS.

8. Класс должен содержать хотя бы один атрибут и хотя бы один метод.

9. Не использовать запрещённые конструкции: eval, exec, subprocess.
10.

Тренажер - 20504-vulnreport

Решение:

```
import json

class VulnReport:
    def __init__(self, data_file="vulnreport.json"):
        """Загружает JSON-файл и хранит данные"""
        self.data = []
        try:
            with open(data_file, "r", encoding="utf-8") as f:
                self.data = json.load(f)
        except FileNotFoundError:
            print(f"Файл {data_file} не найден")
        except json.JSONDecodeError:
            print("Ошибка чтения JSON файла")

    def count_vulnerabilities(self):
        """Возвращает количество уязвимостей"""
        return len(self.data)

    def list_ips(self):
        """Возвращает список всех IP, если они есть"""
        result = []
        for item in self.data:
            ip = item.get("ip") or item.get("IP") or item.get("address")
            if ip:
                result.append(ip)
        return result

    def highest_cvss(self):
        """Возвращает уязвимость с самым высоким CVSS"""
        highest = None
        max_score = -1
        for item in self.data:
            cvss = float(item.get("cvss", -1))
```

```
        if cvss > max_score:
            max_score = cvss
            highest = item
    return highest
```

Критерии оценивания

Критерий	**Баллы**	**Требования**
1. Корректность работы программы	0-1	– 1 балл: программа работает без ошибок, полностью соответствует условиям задачи, и выводит правильный ответ. – 0 баллов: программа не работает или выдает критичные ошибки, выводит неправильный ответ.
2. Использование необходимых функций	0-2	– 2 балла: при решении использованы все требуемые необходимые языковые средства – 1 балл: при решении использованы не все необходимые языковые средства – 0 баллов: требуемые языковые средства не использованы

Шкала оценивания:

Зачтено — 3 балла

Не зачтено — 0 – 2 балла

Требования к оформлению результата выполнения задания: решение задачи предусматривает ввод программного кода в специальное поле тренажера.